pattern for first-party clients, because the ID token can be used directly as a stateless session token. For example, the Natter web UI could use OIDC to authenticate a user and then store the ID token as a cookie or in local storage. The Natter API would then be configured to accept the ID token as a JWT, verifying it with the public key from the OP. An ID token is not appropriate as a replacement for access tokens when dealing with third-party clients for the following reasons:

- ID tokens are not scoped, and the user is asked only for consent for the client to access their identity information. If the ID token can be used to access APIs then any client with an ID token can act as if they are the user without any restrictions.
- An ID token authenticates a user to the client and is not intended to be used by that client to access an API. For example, imagine if Google allowed access to its APIs based on an ID token. In that case, any web site that allowed its users to log in with their Google account (using OIDC) would then be able to replay the ID token back to Google's own APIs to access the user's data without their consent.
- To prevent these kinds of attacks, an ID token has an audience claim that only lists the client. An API should reject any JWT that does not list that API in the audience.
- If you're using the implicit or hybrid flows, then the ID token is exposed in the URL during the redirect back from the OP. When an ID token is used for access control, this has the same risks as including an access token in the URL as the token may leak or be stolen.